A

Mini Project Report Of

PCC-201-COM: Data Structures

Submitted by: -

Parshwa Sankpal (54)

Under the Guidance of

Prof. Mrs. Tejal Patil



BHARATI VIDYAPEETH'S
COLLEGE OF ENGINEERING, LAVALE, PUNE, 412115

Department of Computer Engineering

Savitribai Phule Pune University
Academic Year 2026-27

BHARATI VIDYAPEETH

COLLEGE OF ENGINEERING, LAVALE, PUNE

CERTIFICATE

This is to certify that the "**Mini Project Report**" submitted by **Parshwa Sankpal (54),** on **"Implementation of Keyword Frequency Counter [Hash Table]"** is work done by her and submitted during 2026–2027 academic year, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in computer Engineering, at **Bharati Vidyapeeth's College of Engineering, Lavale, Pune**.

Mrs. Tejal Patil
(Guide)

Dr. Lalit Patil
Head of Computer Engg. Department

ACKNOWLEDGEMENT

It gives immense pleasure to express our gratitude & thanks to all those who have helped in the timely completion of the Mini-Project work. We have a great pleasure in expressing our deep sense of gratitude and indebtedness to **Mrs. Tejal Patil** Guide for their invaluable guidance, constant supervision and sustained encouragement throughout the tenure in this Mini Project work.

We would like to express our thanks to **Dr. Lalit Patil**, Head of Department, Department of Computer Engineering Bharati Vidyapeeth's College of Engineering, Lavale, Pune for their constant support and encouragement.

We would like to express our thanks to **Prof. Dr. R. N. Patil** Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for permitting us to take up the project work. We are thankful to our parents and friends for their kind help and support throughout the course.

Parshwa Sankpal

ABSTRACT

The **Frequency Counting with Hash Tables** project is a Python-based application designed to count the frequency of words in a text file. The system reads the input file, processes the words by converting them into lowercase and removing punctuation, and stores each word along with its frequency.

A **Hash Table** is used for efficient storage and searching of words. A **Hash Function** generates an index for each word, while **Linked List Chaining** is used to handle collisions when multiple words are assigned to the same index. If a word already exists in the Hash Table, its frequency is increased; otherwise, a new node is created.

The project demonstrates the practical application of important **Data Structures concepts**, including Hash Tables, Hash Functions, Linked Lists, Nodes, and Collision Handling. It provides an efficient and simple approach for analyzing word frequencies in text data.

Acknowledgement i
Abstract ii
Content iii
List Of Figures iv

TABLE OF CONTENTS

SR.NO.	CONTENT	PAGE NO.
1	**Introduction** a) Overview b) Problem Statement c) Objective	1
2	**Literature Survey**	3
3	**System Requirement**	4
4	**UML Diagram**	6
5	**System Design** a) Existing system b) Proposed system c) ER Diagram d) System code	7
6	**Result**	13
7	**Application**	14
8	**Future Scope**	15
9	**Conclusion**	16
10	**References**	17

LIST OF FIGURES

Figure	Description	Page No.
Fig 4.1	UML Diagram	6
Fig 5.1	System Code	7
Fig 5.2	System Design	8
Fig 5.3	Result	13

1. Introduction

a) Overview

Frequency Counting with Hash Tables is a Python-based Data Structures project that counts the frequency of each word in a text file. The system reads the input .txt file, removes punctuation, converts words into lowercase, and stores each word with its frequency.

A **Hash Table** is used for efficient storage and searching of words. **Linked Lists** are used to handle collisions using the **chaining technique**. This project demonstrates the practical use of Hash Table, Hash Function, Linked List, Nodes, and Collision Handling.

b) Problem Statement

Counting the frequency of words in a large text file using simple searching methods can require repeated comparisons and may become inefficient.

The problem is to develop a system that can:

- Read words from a text file.
- Convert words into a standard lowercase format.
- Remove punctuation from words.
- Store each unique word in a Hash Table.
- Count how many times each word occurs.
- Handle hash collisions using Linked Lists.
- Display the word and its frequency.

c) Objective

The main objectives of this project are:

1. To develop a **word frequency counting system** using Python.
2. To understand and implement a **Hash Table**.
3. To implement a **Hash Function** for generating table indexes.
4. To handle collisions using **Linked List chaining**.
5. To count and display the frequency of each word.

6. To understand the practical application of **Hash Tables and Linked Lists** in Data Structures.

2. Literature Survey

The literature survey helps to understand existing concepts and techniques related to **word frequency counting, Hash Tables, Hash Functions, Linked Lists, and collision handling**.

2.1 Hash Table

Hash Tables are commonly used for storing and retrieving data efficiently. A key is converted into an index using a hash function, allowing fast searching and insertion.

2.2 Hash Function

A hash function converts a key, such as a word, into an index of the Hash Table. A good hash function distributes data across the table and reduces collisions.

2.3 Collision Handling

A collision occurs when two different words produce the same hash-table index. **Chaining** is a common technique for handling collisions, where multiple elements are stored using a linked list at the same index.

2.4 Linked List

A Linked List consists of nodes connected through links. In this project, linked lists are used to store multiple words that have the same hash index.

2.5 Word Frequency Counting

Word frequency counting is used to determine how many times each word occurs in a text. It is commonly used in text processing, document analysis, search systems, and Natural Language Processing.

2.6 Summary of Literature Survey

From the study of these concepts, it is observed that **Hash Tables provide efficient data storage and searching**, while **Linked Lists provide an effective way to handle collisions**. Combining these data structures provides a suitable approach for implementing a word frequency counting system.

3. System Requirements

The following hardware and software requirements are needed to develop and run the **Frequency Counting with Hash Tables** project.

3.1 Hardware Requirements

Requirement	**Specification**
Processor	Intel Core i3 or above
RAM	Minimum 4 GB
Storage	Minimum 500 MB free space
Input Device	Keyboard and Mouse
Display	Standard Monitor
Operating System	Windows / Linux / macOS

3.2 Software Requirements

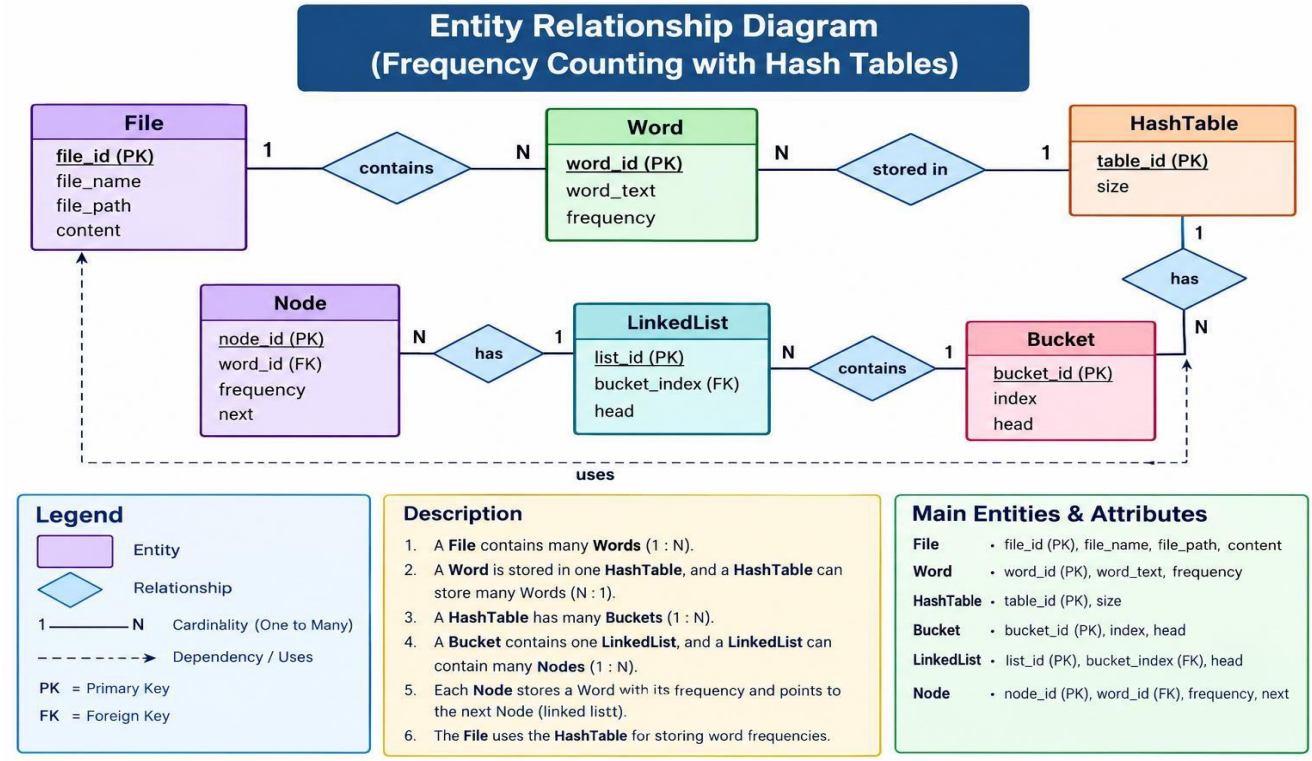
Software	**Requirement**
Programming Language	Python 3.x
IDE / Code Editor	VS Code / PyCharm / IDLE
Operating System	Windows / Linux / macOS
Input File	.txt text file
Libraries	Python Standard Library

3.3 Functional Requirements

The system should be able to:

1. Read text from an input file.
2. Process and normalize the words.
3. Generate a hash index for each word.
4. Store words in the Hash Table.
5. Handle collisions using Linked Lists.
6. Increase the frequency when a word already exists.
7. Display each word with its frequency.
8. Test the correctness of frequency counting.

4. UML DIAGRAM



5. System Design

a) Existing System

In the existing system, word frequency can be counted using simple searching or by repeatedly comparing each word with previously stored words.

The main limitations of the existing approach are:

- Words may need to be searched repeatedly.
- Processing becomes slower as the number of words increases.
- Duplicate words require repeated comparisons.
- Collision handling is not specifically implemented.
- It does not demonstrate the practical use of Hash Tables and Linked Lists.
- Managing a large number of words can become inefficient.

b) Proposed System

The proposed system uses a **Hash Table with Linked List Chaining** to count the frequency of words in a text file.

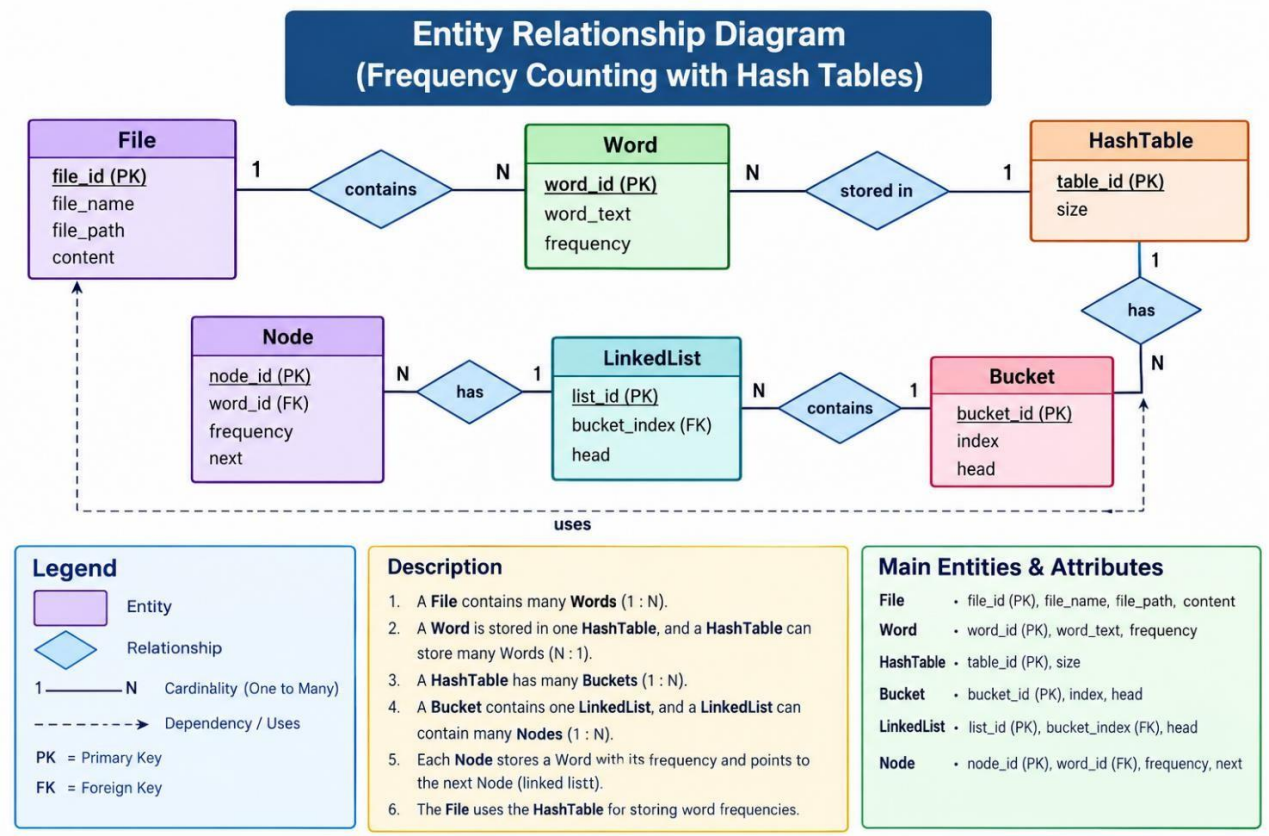
Working of Proposed System

1. The user enters the name of a .txt file.
2. The system checks whether the file exists.
3. Words are read from the file.
4. Punctuation is removed and words are converted to lowercase.
5. A Hash Table of size **8** is created.
6. A Hash Function generates an index for each word.
7. The word is searched in the corresponding Linked List.
8. If the word already exists, its frequency is increased by 1.
9. If the word does not exist, a new Node is created.
10. If multiple words have the same index, **chaining using Linked Lists** handles the collision.

11. Finally, all words and their frequencies are displayed. ↓

c) ER Diagram

The ER diagram represents the main entities and their relationships in the system.



d) System Code

The project is implemented using **Python**. It consists mainly of main.py, HashTable.py, LinkedList.py, and Node.py.

1. Main Program – main.py

```
from HashTable import HashTable

from os import path

import string

print("Welcome to Word Frequency Counter!")
```

```
filename = input("What is the name of the .txt file? ")
while path.exists(filename) == False:
    print("File does not exist. Try again.")
    filename = input("What is the name of the .txt file? ")
words_in_file = []
with open(filename, 'r') as file:
    for line in file:
        for word in line.split():
            word = word.strip(string.punctuation)
            words_in_file.append(word.lower())
frequency_counter = HashTable(8)
for word in words_in_file:
    frequency_counter.insert(word, 1)
frequency_counter.print_key_values()
```

2. Hash Table – HashTable.py

```
from LinkedList import LinkedList
class HashTable:


    def __init__(self, size):
        self.size = size
        self.arr = self.create_arr(size)
    def create_arr(self, size):
        array = []
        for i in range(size):
            newLinkedList = LinkedList()
            array.append(newLinkedList)
```

```
        return array
    def hash_func(self, key):
        word = key.lower()
        integer_from_string = ord(word[0])
        index = integer_from_string % self.size
        return index
    def insert(self, key, value):
        hash_key = self.hash_func(key)
        newLinkedList = self.arr[hash_key]
        found = newLinkedList.find(key)
        if found == -1:
            item = (key, value)
            self.arr[hash_key].append(item)
    def print_key_values(self):
        for LinkedList in self.arr:
            LinkedList.print_nodes()
```

3. Linked List – LinkedList.py

```
from Node import Node
class LinkedList:
    def __init__(self):
        self.head = None
    def append(self, new_data):
        new_node = Node(new_data)
        new_node.next = self.head
        self.head = new_node
    def find(self, key):
```

```
        current = self.head
        found = False
        while current != None and not found:
            if current.data[0] == key:
                found = True
            else:
                current = current.next
        if found:
            new_tuple = (key, current.data[1] + 1)
            current.data = new_tuple
        else:
            return -1
    def print_nodes(self):
        current = self.head
        while current != None:
            print(
                f'{current.data[0]} : {current.data[1]}'
            )
            current = current.next
```

4. Node – Node.py

```
class Node:


    def __init__(self, data):
        self.data = data
        self.next = None
```

Technologies Used

- **Programming Language:** Python
- **Data Structure:** Hash Table
- **Collision Handling:** Chaining
- **Supporting Data Structure:** Linked List
- **Input:** Text File (.txt)
- **Hash Function:** ASCII value using ord() and modulo operator %

6. Result

```
[3]: %run main.py

Welcome to Word Frequency Counter! 📊
📝 What is the name of the .txt file?  example.txt


✅ We found the following 47 words in example.txt:

['cricket', 'is', 'a', 'very', 'popular', 'sport', 'played', 'with', 'a', 'bat', 'and', 'a', 'ball', 'millions', 'of', 'fans', 'watch', 'cricket', 'matc
hes', 'around', 'the', 'world', 'players', 'work', 'hard', 'to', 'win', 'matches', 'for', 'their', 'teams', 'winning', 'a', 'cricket', 'match', 'bring
s', 'great', 'joy', 'to', 'the', 'players', 'and', 'the', 'fans', 'who', 'support', 'them']


⬇️ Now adding 47 words from example.txt to the Frequency Counter...


🖨️ All done! Here are all of the words from example.txt and their frequencies:

hard : 1
players : 2
played : 1
popular : 1
around : 1
and : 2
a : 4
is : 1
joy : 1
brings : 1
ball : 1
bat : 1
support : 1
sport : 1
cricket : 3
them : 1
teams : 1
their : 1
to : 2
the : 3
match : 1
matches : 2
millions : 1
for : 1
fans : 2
very : 1
who : 1
great : 1
winning : 1
win : 1
work : 1
world : 1
watch : 1
of : 1
with : 1
```

7.APPLICATIONS

The **Frequency Counting with Hash Tables** system can be used in various areas where word frequency needs to be calculated.

- **Text Analysis:** Counts the occurrence of words in documents.
- **Search Engines:** Helps analyze frequently used search terms.
- **Natural Language Processing:** Useful for basic text and language analysis.
- **Document Processing:** Can identify commonly used words in documents.
- **Data Analysis:** Helps summarize and analyze textual data.
- **Educational Applications:** Demonstrates practical implementation of Hash Tables and Linked Lists.

8. Future Scope

The project can be improved and extended in the future in the following ways:

1. The system can support **large text files** efficiently.
2. A **Graphical User Interface (GUI)** can be added for easier use.
3. The system can display frequencies using **charts and graphs**.
4. Multiple text files can be processed simultaneously.
5. The Hash Table size can be dynamically increased according to the input.
6. More efficient hash functions can be implemented.
7. The system can be extended for **basic Natural Language Processing (NLP)** applications.
8. Stop-word removal and advanced text preprocessing can be added.

9. Conclusion

The **Frequency Counting with Hash Tables** project successfully demonstrates the practical implementation of **Hash Tables, Hash Functions, Linked Lists, Nodes, and Collision Handling** using Python.

The system reads words from a text file, processes them, stores them in a Hash Table, and calculates the frequency of each word. **Linked List chaining** is used to handle collisions between words having the same hash index.

This project provides a practical understanding of how Hash Tables can be used for efficient data storage, searching, and frequency counting.

10. References

1. **Python Documentation** – Python Programming Language and File Handling.
2. **GeeksforGeeks** – Hashing, Hash Tables, Linked Lists, and Collision Handling.
3. **W3Schools** – Python Programming and Data Structures.
4. **Data Structures and Algorithms** – Concepts of Hashing and Linked Lists.
5. Project source code and `README.md` documentation.